

Finding Optimal Cyclic Quorum Systems by Combinatorial Search and Reinforcement Learning

Wai-Shing Luk

Abstract—This paper studies two computational approaches to finding optimal Cyclic Quorum Systems (CQS). A CQS is a symmetric quorum system in which the whole family of quorums is generated by cyclically shifting a single base quorum, so that equal work and equal responsibility hold automatically. The central difficulty is to identify that base quorum, a problem equivalent to finding a difference cover, or relaxed difference set.

The first approach is a systematic combinatorial search. It uses generate-and-test with pruning and backtracking, together with symmetry breaking inspired by algorithms for generating fixed-density necklaces and bracelets, and it is guaranteed to find a solution within its search space if one exists. The second approach applies deep reinforcement learning (RL) and treats the construction as a sequential decision process; unlike the combinatorial method, it is not guaranteed to find a solution within a bounded number of episodes.

Both methods use parallelism to improve efficiency. The recursive search divides the search space into partitions, each handled by a separate thread, while the RL method runs several agents that learn independently but share a single neural network.

I. INTRODUCTION

Distributed systems must coordinate many independent sites while using resources efficiently. A quorum system is one of the basic tools for this: it is a collection of subsets, called quorums, of the set of sites, in which any two quorums intersect. The intersection property guarantees that any two operations requiring a quorum meet at a common site, which makes conflict detection and resolution possible; it underlies applications such as mutual exclusion and the reliable processing of distributed data.

Designing quorum systems that are both fair and scalable, however, is difficult. Ideally each quorum has the same size (equal work) and each site occurs in the same number of quorums (equal responsibility), and the quorum size is as small as possible in order to reduce communication cost. For N sites satisfying these symmetry conditions the quorum size is approximately $\sqrt{N}$. Attaining this bound is often equivalent to constructing a finite projective plane, and such planes do not exist for every order, so finding optimal quorums is a hard combinatorial problem.

Cyclic quorum systems (CQS) are a structured construction that often achieves, or comes close to, these properties. They draw on cyclic block designs and cyclic difference sets, and their defining feature is that the whole family of quorums is generated cyclically from a single base

quorum, which simplifies both their description and their management.

Cyclic quorum systems are useful beyond their original application in distributed mutual exclusion. Recent work applies them to distributed all-pairs computations (Kleinheksel and Somani 2016, 2018), to heterogeneous wakeup scheduling in wireless sensor networks (Lai et al. 2010; Jiang et al. 2005), and to sequence parallelism for attention computation in transformers (Bian and Somani 2025).

The main objective of this paper is to apply difference covers to the construction of optimal cyclic quorum systems. A difference cover is a set of numbers whose pairwise differences, taken modulo N , cover every non-zero residue at least once and, ideally, without excessive repetition; the problem is to select exactly d numbers from $\{0, \dots, N-1\}$. Earlier work tabulated base quorums for optimal cyclic quorum schemes with N from 4 to 111 using 1-based indexing; for consistency, this paper reports every base quorum with 0-based residues and extends the search to $N = 150$ and beyond.

This paper considers two computational approaches to finding these structures: a systematic recursive search and a trial-and-error method based on deep reinforcement learning. Both use parallel processing.

The rest of the paper is organized as follows. Related work is reviewed next, after which we recall quorum systems and cyclic quorum systems, work through a small construction in detail, and survey applications. We then state the difference cover problem and present the two approaches—a combinatorial search and a deep reinforcement learning method—before comparing them, reporting implementation and performance, and concluding. The appendix details the fixed-density bracelet generator used for symmetry breaking.

II. RELATED WORK

Quorum systems coordinate distributed processes without global knowledge or central control, and permission-based mutual exclusion algorithms built on them are surveyed by Saxena and Rai (Saxena and Rai 2003). Maekawa’s algorithm was among the first to pursue symmetry—equal work and equal responsibility—with a minimal quorum size, and it established the connection to finite projective planes (Maekawa 1985). Finding optimal symmetric quorums for arbitrary N , however, proved difficult.

Luk and Wong proposed using cyclic difference sets as the basis for quorum systems in distributed mutual exclusion (Luk and Wong 1997). Their cyclic scheme constructs symmetric quorum sets for arbitrary N with a quorum size very close to the theoretical lower bound $\sqrt{N}$. The construction studied in this paper is a direct descendant of that scheme, and its main computational challenge is to find the optimal base quorum, that is, a relaxed difference set.

Cyclic quorums have since been applied well beyond mutual exclusion: to distributed all-pairs computations (Kleinheksel and Somani 2016, 2018), to heterogeneous wakeup scheduling in wireless sensor networks (Lai et al. 2010), and to sequence parallelism for attention (Bian and Somani 2025). More generally, quorum-based rendezvous and neighbor-discovery protocols are used in cognitive-radio, Internet-of-Things, and underwater acoustic networks (Bian et al. 2009, 2012; Bian and Park 2011; Pozza et al. 2015; Bouabdallah et al. 2018; Huang et al. 2025).

III. PRELIMINARIES

A. Quorum Systems

A quorum system coordinates a distributed system without global knowledge or central control. Let $U = \{0, 1, \dots, N-1\}$ be the set of N sites (or processes).

Definition 1. Quorum System. A quorum system $\mathcal{Q}$ over U is a collection of non-empty subsets of U , called quorums, which satisfies the **intersection property**: $G \cap H \neq \emptyset$ for all $G, H \in \mathcal{Q}$.

This property is essential: it guarantees that any two operations requiring a quorum consult at least one common site, which can mediate conflicts or ensure consistency (Saxena and Rai 2003).

For distributed algorithms such as mutual exclusion, further properties are desirable for fairness and efficiency:

- **A1. Site Inclusion:** Each site i is contained in its own quorum S_i .
- **A2. Non-empty Intersection:** Any two quorums must intersect. This is the defining property of a quorum system.
- **A3. Equal Work:** All quorums have the same size: $|S_i| = d$ for every site i , where d is an integer with $3 \leq d < N$.
- **A4. Equal Responsibility:** Each site is contained in the same number d of quorums.

A set of quorums satisfying A3 and A4 is called **sym-metric**. The goal in designing efficient quorum systems is to minimize the quorum size d while maintaining these properties.

Maekawa showed that for a fixed quorum size d , the maximum number of sites for which a symmetric quorum system exists is $d(d-1)+1$, assuming any two quorums intersect at exactly one site (Maekawa 1985). This implies

a theoretical lower bound on the quorum size: $d \geq \sqrt{N}$. More precisely, the lower bound is $\lceil \sqrt{N} \rceil$. Grid-based constructions are simpler to describe, but their quorums typically have size about $2\sqrt{N}-1$, roughly twice this bound.

Finding systems that satisfy these properties with a near-minimal size is difficult. For $N = d(d-1)+1$ the problem is equivalent to constructing a finite projective plane of order $d-1$, and such planes do not exist for every order (order 10, for instance). Exhaustive search is intractable for large N , which has motivated systematic constructions based on combinatorial designs such as difference sets.

The difficulty of finding optimal symmetric quorum systems for arbitrary N highlights the value of structured constructions that exploit combinatorial structure, such as the cyclic quorum systems discussed next.

B. Cyclic Quorum Systems

Cyclic quorum systems construct symmetric quorum systems from cyclic block designs and cyclic difference sets. Their defining feature is that the entire collection of quorums is obtained by cyclically shifting the elements of a single base quorum.

Let $U = \{0, 1, \dots, N-1\}$ be the set of N sites, viewed as elements of the additive group $\mathbb{Z}_N$.

Definition 2. Cyclic Quorum System. A group of cyclic quorums is a set of N quorums $\{B_0, B_1, \dots, B_{N-1}\}$ where each quorum B_i is derived from a base quorum $B_0 = \{a_0, a_1, \dots, a_{d-1}\} \subseteq \{0, 1, \dots, N-1\}$ by adding i modulo N to each element: $B_i = \{a_0+i, a_1+i, \dots, a_{d-1}+i\} \pmod{N}$.

For $\{B_0, \dots, B_{N-1}\}$ to be a valid quorum system it must satisfy the non-empty intersection property, $B_i \cap B_j \neq \emptyset$ for all $i, j \in \{0, \dots, N-1\}$. The cyclic construction automatically gives equal work ($|B_i| = d$ for every i) and equal responsibility (each element occurs in d quorums).

A cyclic quorum system also satisfies the stronger **rotation-closure** property: for all quorums G, H and every integer i , $G \cap (H+i) \neq \emptyset$, where $H+i$ denotes the cyclic shift of H by i .

The existence and properties of cyclic quorum systems are deeply connected to the theory of **difference sets** (Hall 1986).

Definition 3. Cyclic (N, d, λ) -Difference Set. A set $M = \{a_0, a_1, \dots, a_{d-1}\} \subseteq \{0, 1, \dots, N-1\}$ is called a cyclic (N, d, λ) -difference set if for every $m \not\equiv 0 \pmod{N}$, there are exactly λ ordered pairs (a_i, a_j) with $a_i, a_j \in M$ such that $a_i - a_j \equiv m \pmod{N}$.

A related concept, particularly relevant for CQS, is the relaxed difference set.

Definition 4. Relaxed Cyclic (N, d) -Difference Set. A set $M = \{a_0, a_1, \dots, a_{d-1}\} \subseteq \{0, 1, \dots, N-1\}$ is called

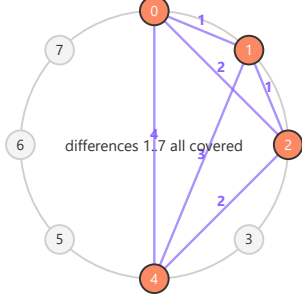


Fig. 1. The base quorum $\{0, 1, 2, 4\}$ on a circle of $N = 8$ points. Each chord spans one difference; together the chords cover every non-zero residue modulo 8.

a relaxed (cyclic) (N, d) -difference set if for every $m \not\equiv 0 \pmod{N}$, there exists at least one ordered pair (a_i, a_j) with $a_i, a_j \in M$ such that $a_i - a_j \equiv m \pmod{N}$.

More generally, a difference cover with parameters (v, d, λ) is a multiset D of elements of a finite abelian group G in which every $z \in G$ occurs exactly λ times as a non-trivial difference $z = x_i - x_j$ with $x_i, x_j \in D$. When G is cyclic the cover is called cyclic, which is the case relevant here.

The connection between cyclic quorum systems and relaxed difference sets is standard. A set $B_0 = \{a_0, \dots, a_{d-1}\} \pmod{N}$ is a relaxed (N, d) -difference set if and only if the cyclic quorum set defined by $B_i = \{a_0 + i, \dots, a_{d-1} + i\} \pmod{N}$ satisfies the non-empty intersection property. The intuition is that the intersection property $B_i \cap B_j \neq \emptyset$ for all i, j is equivalent to requiring that for every difference $m = j - i \pmod{N}$ there exist elements $a_u, a_v \in B_0$ such that $a_u + i \equiv a_v + j \pmod{N}$, which simplifies to $a_u - a_v \equiv j - i \equiv m \pmod{N}$. Thus every non-zero difference modulo N must be formed by at least one pair of elements of the base quorum.

Finding a base quorum B_0 that forms a cyclic quorum system of minimum size d is equivalent to finding a relaxed (N, d) -difference set with minimum d . For $N = d(d-1) + 1$ with $d-1$ a prime power, the existence of cyclic $(N, d, 1)$ -difference sets (Singer difference sets) is guaranteed, and these sets form cyclic quorum systems in which any two quorums intersect in exactly one site (Hall 1986). Singer difference sets therefore lead to particularly efficient CQS with no redundancy in pair coverage. For other values of N , finding the optimal base quorum often requires exhaustive search, although the Multiplier Theorem can speed up the construction of certain difference sets by identifying primes p under which multiplication permutes the elements of the set.

Examples of cyclic quorum systems and difference sets include:

- For $N = 7$, the set $\{1, 2, 4\} \pmod{7}$ is a cyclic $(7, 3, 1)$ -difference set. This is a Singer difference set ($7 =$

$3(3-1) + 1$, and $3-1 = 2$ is a prime power). The cyclic quorums are $\{1, 2, 4\}, \{2, 3, 5\}, \{3, 4, 6\}, \{4, 5, 0\}, \{5, 6, 1\}, \{6, 0, 2\}, \{0, 1, 3\}$.

- For $N = 8$, the set $\{0, 1, 2, 4\} \pmod{8}$ is a base quorum. The quorums are $B_0 = \{0, 1, 2, 4\}$, $B_1 = \{1, 2, 3, 5\}$, $B_2 = \{2, 3, 4, 6\}$, $B_3 = \{3, 4, 5, 7\}$, $B_4 = \{4, 5, 6, 0\}$, $B_5 = \{5, 6, 7, 1\}$, $B_6 = \{6, 7, 0, 2\}$, $B_7 = \{7, 0, 1, 3\}$.
- Earlier work tabulated base quorums for optimal cyclic quorum schemes with N from 4 to 111 using 1-based indexing, found through exhaustive search. For consistency, this paper reports every base quorum with 0-based residues; for example, the base quorums for $N = 4$ and $N = 10$ are $\{0, 1, 2\}$ and $\{0, 1, 2, 5\}$, respectively.

The existence of cyclic quorum systems with sizes close to the theoretical lower bound for arbitrary N , coupled with their symmetric properties, makes them attractive for managing distributed resources and computations.

IV. A WORKED EXAMPLE: A CYCLIC QUORUM SYSTEM FOR $N = 8$

We construct a cyclic quorum system for $N = 8$ from the base quorum $B_0 = \{0, 1, 2, 4\} \pmod{8}$, of size $d = 4$. The full set of 8 quorums is generated by adding $i \in \{0, 1, \dots, 7\}$ modulo 8 to the elements of B_0 :

- $B_0 = \{0, 1, 2, 4\} \pmod{8}$
- $B_1 = \{0 + 1, 1 + 1, 2 + 1, 4 + 1\} \pmod{8} = \{1, 2, 3, 5\} \pmod{8}$
- $B_2 = \{0 + 2, 1 + 2, 2 + 2, 4 + 2\} \pmod{8} = \{2, 3, 4, 6\} \pmod{8}$
- $B_3 = \{0 + 3, 1 + 3, 2 + 3, 4 + 3\} \pmod{8} = \{3, 4, 5, 7\} \pmod{8}$
- $B_4 = \{0 + 4, 1 + 4, 2 + 4, 4 + 4\} \pmod{8} = \{4, 5, 6, 0\} \pmod{8}$
- $B_5 = \{0 + 5, 1 + 5, 2 + 5, 4 + 5\} \pmod{8} = \{5, 6, 7, 1\} \pmod{8}$
- $B_6 = \{0 + 6, 1 + 6, 2 + 6, 4 + 6\} \pmod{8} = \{6, 7, 0, 2\} \pmod{8}$
- $B_7 = \{0 + 7, 1 + 7, 2 + 7, 4 + 7\} \pmod{8} = \{7, 0, 1, 3\} \pmod{8}$

These eight quorums form a cyclic quorum system for $N = 8$. Each has size 4 (equal work). Equal responsibility also holds: site 0 occurs in B_0, B_4, B_6 and B_7 , that is, in four quorums, matching $d = 4$; by the cyclic construction the same is true of every site. Intersection can be checked on any pair, for instance $B_0 \cap B_1 = \{1, 2\} \neq \emptyset$.

The challenge is to find the base quorum B_0 for a given N such that the resulting cyclic sets satisfy the intersection property and d is minimal.

V. APPLICATIONS

Cyclic quorum systems and difference covers are used well beyond mutual exclusion.

	0	1	2	3	4	5	6	7
B0	0	1	2		4			
B1		1	2	3		5		
B2			2	3	4		6	
B3				3	4	5		7
B4	0				4	5	6	
B5		1				5	6	7
B6	0		2				6	7
B7	0	1		3				7

Fig. 2. Quorum matrix for $N = 8$ with base quorum $\{0, 1, 2, 4\}$. Row B_i lists the members of quorum B_i ; the cyclic shifts form a diagonal band.

Distributed mutual exclusion. A site requests access from every member of its quorum; the members check for conflicts, and the site enters the critical section only when they agree. The quorum size approaches $\sqrt{N}$, and the cyclic construction provides symmetry while requiring only one base quorum.

Distributed all-pairs algorithms. Each process stores only a quorum of the data, and the all-pairs property guarantees that every pair of data sets occurs together in some quorum. The quorum size grows as $O(\sqrt{N})$ in the number of processes N , which reduces memory substantially; reported experiments give up to a sevenfold speedup on eight nodes with a two-thirds reduction in memory (Kleinheksel and Somani 2016, 2018).

Wireless sensor networks. A variant called CQS-Pair combines two cyclic quorum systems with different cycle lengths to satisfy a heterogeneous rotation-closure property, so that nodes with different power-saving requirements are guaranteed to discover one another within every m consecutive slots. This balances energy consumption against discovery delay (Lai et al. 2010).

Attention in deep learning. CQS-Attention applies the same theory to scale the standard self-attention computation to very long sequences, easing the memory bottleneck (Bian and Somani 2025).

String algorithms. Difference covers also underlie linear-time suffix-array construction: the DC3 algorithm samples positions by residue modulo 3 and uses the difference-cover property that any two positions share a small offset at which both are sample positions, which makes the comparison and merging steps efficient (Kärkkäinen et al. 2006; Mehnert 2024).

VI. THE DIFFERENCE COVER PROBLEM

Given the range 0 to $N - 1$ and a required size d , the problem is to choose d numbers such that the positive

differences between every pair, taken modulo N , include every value from 1 to $N - 1$ at least once. Equivalently, place d markers on a circle of N points so that the pairwise distances cover every distance around the circle.

We assume throughout that $N, d \geq 3$ and that $N \leq d(d - 1) + 1$.

VII. COMBINATORIAL SEARCH

The first approach is a **systematic algorithm** for finding difference covers, following a **generate-and-test strategy with pruning**. Solutions are **built incrementally**, one number at a time. Whenever a number is added, the algorithm computes its differences with the numbers already chosen and updates the set of differences seen so far.

A. Search with Pruning and Backtracking

A critical technique is **backtracking**: when the algorithm reaches a **dead end**—a partial solution that cannot lead to a valid complete solution—it undoes the last choice and tries a different one. The **StepForward** and **StepBackward** routines maintain the difference counts as the search descends and backtracks through the space.

Optimization avoids spending time on impossible partial solutions. Before descending further, the algorithm checks whether the partial set can still cover enough distinct differences to reach a valid complete solution; if not, the branch is pruned. Two structures track coverage: an array recording, for each residue, how often it has been produced, and a counter of how many distinct residues have been covered. The difference of an element with itself is 0 and is covered from the outset.

Concretely, when a new element a_t is appended, **StepForward** compares it with each previously chosen element a_j ($j < t$), computes the two distances $a_t - a_j$ and $N - (a_t - a_j)$, and records the smaller one; the distinct-difference counter is incremented the first time a distance appears. **StepBackward** mirrors this, decrementing the counts when the element is removed and restoring the state that preceded its insertion.

Symmetry between mirror images is removed by a reflection test. The acceptance test is applied when $d - 1$ elements have been placed: the final element is forced, and the configuration is reported only if the distinct differences cover every non-zero residue. Because a difference m and its complement $N - m$ arise from the same pair in opposite order, it suffices to track distances up to $N/2$; covering all of them covers every non-zero difference modulo N .

B. Symmetry Breaking by Bracelet Generation

The reflection test is the fixed-density **bracelet** condition. A candidate base quorum corresponds to a binary string of length N with d non-zero positions; its rotations form a necklace, and its rotations together with its reversal form a bracelet. Enumerating one representative per rotation

Algorithm 1 Recursive search for a difference cover

```

1: function SEARCH( $N, d$ )
2:    $a[1] \leftarrow 0$ ;  $cnt \leftarrow 0$ ;  $diff[\cdot] \leftarrow 0$ ;  $diff[0] \leftarrow 1$ 
3:   RECURSE(1, 1)
4: end function
5: function RECURSE( $t, cnt$ )
6:   if  $t = d$  then
7:     REPORT( $a$ ); return
8:   end if
9:   for  $v \leftarrow a[t] + 1$  to  $N - d + t + 1$  do
10:     $a[t + 1] \leftarrow v$ 
11:     $cnt' \leftarrow$  STEPFORWARD( $t + 1, cnt$ )
12:    if  $cnt' + free(t + 1) \geq N - 1$  then
13:      if CHECKREV( $t + 1$ )  $\neq -1$  then
14:        RECURSE( $t + 1, cnt'$ )
15:      end if
16:    end if
17:    STEPBACKWARD( $t + 1$ )
18:  end for
19: end function
20: function STEPFORWARD( $t, cnt$ )
21:  for  $j \leftarrow 1$  to  $t - 1$  do
22:     $\delta \leftarrow \min(a[t] - a[j], N - (a[t] - a[j]))$ 
23:    if  $diff[\delta] = 0$  then
24:       $cnt \leftarrow cnt + 1$ 
25:    end if
26:     $diff[\delta] \leftarrow diff[\delta] + 1$ 
27:  end for
28:  return  $cnt$ 
29: end function

```

▷ coverage bound: else prune
 ▷ reflection symmetry

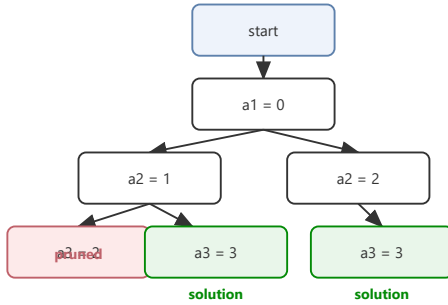


Fig. 3. Depth-first search for a difference cover with $N = 5$ and $d = 3$. A branch that cannot cover all differences is pruned (red); a complete branch is a solution (green).

class (a necklace) removes rotational duplicates, and enumerating one per rotation-and-reflection class (a bracelet) additionally removes reflected duplicates. To do so without losing the efficiency of the search, we use a generator of fixed-density bracelets that runs in **constant amortized time (CAT)**, so the symmetry test adds no asymptotic overhead; it merges the fixed-density necklace construction of Ruskey and Sawada with the bracelet construction of Sawada, and is summarized in Appendix A. Reflection

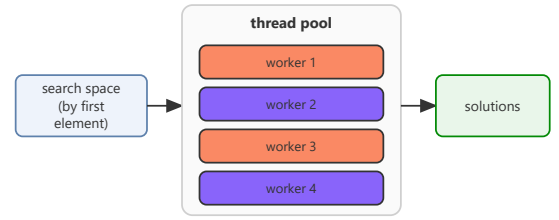


Fig. 4. Parallel search. The search space is partitioned by the value of the first element and distributed over a thread pool.

symmetry only helps near the symmetric frontier of the search, so the measured speedup is modest.

C. Parallel Search

The search is accelerated by a **thread pool**. Different threads start from different values of the first non-zero element, partitioning the search space; each worker runs an independent depth-first search on its partition and reports any solution it finds. Multiple cores therefore work on disjoint partitions simultaneously, and the main thread waits for the workers to complete, displaying a countdown of outstanding tasks.

D. Results: Optimal Base Quorums

The recursive search is **exact**: within its search space it returns a solution whenever one exists, using explicit pruning rules derived from the coverage lower bound, and the returned base quorum is therefore optimal. Table I lists the optimal base quorums found for $N = 112$ to 150; these extend, with 0-based residues, the earlier tabulation for $N = 4$ to 111.

VIII. DEEP REINFORCEMENT LEARNING

The second approach casts the construction as a **sequential decision problem** and solves it with **reinforcement learning**: a policy is learned by trial and error from a reward signal rather than derived from combinatorial rules. It is not guaranteed to find a solution within a bounded number of episodes.

A. Policy Network

The policy is a feed-forward network π_θ with three weight layers: an input layer of size $2N$, hidden layers of size 256 and 128 with rectified-linear (ReLU) activations, and an output layer of size N that produces one logit per element. Weights and biases are initialized by Xavier initialization. Given a state s , the logits $z = \pi_\theta(s)$ are turned into a distribution over the elements not yet chosen by a masked softmax,

$$\text{Softmax}(z_i) = \frac{e^{z_i - \max(z)}}{\sum_j e^{z_j - \max(z)}},$$

where the subtraction of $\max(z)$ is for numerical stability and the logits of already-chosen elements are set to a very large negative value before the softmax, so that they cannot be selected again.

B. Policy-Gradient Learning

The state of an episode concatenates two binary vectors of length N : one indicating which elements have been chosen, and one indicating which residues have been covered; thus $s \in \mathbb{R}^{2N}$. An episode starts from the empty selection and runs for $d - 1$ steps, after which the last element is forced. At step k the agent samples an element $a_k \sim \pi_\theta(\cdot | s_k)$ and receives the reward r_k equal to the number of residues newly covered by adding a_k . The rewards are aggregated into discounted returns $G_k = \sum_{i \geq k} \gamma^{i-k} r_i$ with discount factor γ , and normalized for stable training. The parameters are updated by policy gradient,

$$\theta \leftarrow \theta + \eta \sum_k G_k \nabla_\theta \log \pi_\theta(a_k | s_k),$$

with learning rate η and an entropy bonus of weight β that discourages premature convergence; we use $\gamma = 0.98$, $\eta = 0.01$, and $\beta = 0.01$.

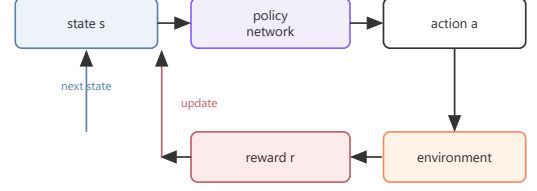


Fig. 5. The reinforcement-learning loop. The policy network selects the next element, the environment returns a reward, and the network is updated by policy gradient.

C. Parallel Agents

As in the recursive search, learning is accelerated by **multi-threading**. Several worker threads run episodes concurrently and share a single policy network, each contributing its gradients to the shared parameters. The shared-network update is lock-free, in the manner of asynchronous (Hogwild-style) stochastic gradient descent, relying on tear-free aligned floating-point stores; only the solution output is protected by a mutex. Each worker continues until some thread finds a solution or the total episode count reaches the limit.

D. Results: Base Quorums beyond the Exhaustive Range

The learned policy is **not guaranteed** to find a solution within the episode budget, and it converges to a solution rather than to all of them. It is nevertheless able to extend the reach of the search beyond the exhaustive range: Table II lists the near-optimal base quorums found for $N = 151$ to 171.

IX. COMPARISON OF THE TWO APPROACHES

Both approaches rely on **parallel processing**—a thread pool over search partitions on one side, and a set of learning agents sharing one network on the other—but their methodologies differ fundamentally. Table III compares the quorum sizes found with the lower bound $\lceil \sqrt{N} \rceil$ for selected N , and Table IV summarizes the qualitative differences.

The recursive search explores the solution space systematically. It uses backtracking and coverage-based pruning, is exhaustive over its partitions, and therefore returns a solution whenever one exists, at the cost of a search space that grows quickly with N . The reinforcement-learning agent instead learns a policy from the reward signal. It is not guaranteed to succeed and it finds one cover rather than all of them, but it remains applicable where the exhaustive search is no longer tractable. Both methods are accelerated by parallelism of different kinds: the search partitions its tree across a thread pool, whereas the learning agents share a single network.

X. IMPLEMENTATION AND PERFORMANCE

Several implementation choices measurably affected performance; Table V summarizes them.

TABLE I
OPTIMAL BASE QUORUMS FOR $N = 112$ TO 150 , USING 0-BASED RESIDUES.

N	Base quorum
112	38 51 55 61 63 75 80 89 96 107 110 111
113	46 48 60 67 75 78 86 89 102 106 111 112
114	56 59 63 67 71 75 79 80 85 110 111 112 113
115	57 60 64 68 72 76 80 81 86 111 112 113 114
116	57 60 63 67 71 75 79 83 88 112 113 114 115
117	37 43 50 51 55 68 70 79 90 100 113 116
118	57 60 62 63 68 74 89 94 98 107 114 116 117
119	57 59 64 69 77 85 93 101 109 112 115 116 118
120	54 55 65 67 74 81 89 96 113 114 117 118 119
121	59 61 66 71 79 87 95 103 111 114 117 118 120
122	54 60 61 68 71 78 90 94 106 110 115 119 121
123	53 55 69 72 74 83 89 96 106 114 118 121 122
124	53 58 70 74 78 85 92 98 111 120 121 122 123
125	54 59 71 75 80 86 93 99 113 121 122 123 124
126	49 51 55 61 64 81 86 91 102 109 110 124 125
127	53 59 63 75 77 82 90 105 115 116 122 125 126
128	52 56 62 65 67 77 91 99 100 107 108 126 127
129	49 57 58 64 67 87 92 97 109 111 113 124 128
130	46 50 58 60 71 84 89 90 99 106 123 126 129
131	55 57 69 76 84 87 96 104 107 120 124 129 130
132	65 68 71 75 79 83 87 91 95 100 128 129 130 131
133	47 56 57 64 77 89 92 103 108 126 130 132
134	66 68 72 75 81 91 101 111 121 125 128 129 132 133
135	67 70 71 75 78 88 98 108 118 120 127 132 133 134
136	67 70 72 76 79 89 99 109 119 120 127 133 134 135
137	68 71 73 77 80 90 100 110 120 121 128 134 135 136
138	64 68 75 82 92 102 112 122 125 128 131 133 134 137
139	64 71 72 78 79 89 99 109 119 122 131 133 135 138
140	61 66 78 82 86 90 97 104 110 127 136 137 138 139
141	67 74 78 85 95 105 115 125 128 131 134 137 139 140
142	62 65 79 81 83 91 94 103 106 133 134 139 140 141
143	56 71 76 77 90 94 98 102 106 130 132 139 141 142
144	55 65 66 68 75 83 97 105 113 117 119 138 140 143
145	58 67 68 78 85 89 93 97 101 125 138 139 141 144
146	58 65 70 78 79 89 92 98 102 127 128 143 144 145
147	63 65 77 84 92 95 103 112 120 123 136 140 145 146
148	55 60 61 79 87 94 105 109 111 125 134 137 146 147
149	55 60 62 76 89 100 106 112 115 125 135 143 147 148
150	50 62 65 68 69 95 97 105 110 118 127 129 143 149

Symmetry breaking. Breaking rotation symmetry alone (necklaces) was extended with the reflection test **CheckRev** of the search to enumerate bracelets. The gain is modest, $1.07\text{--}1.17\times$, because reflection only helps near the symmetric frontier; the output was verified to be identical to a reference solver on twelve parameter pairs with N from 15 to 105.

Learning efficiency. Caching the network activations during an episode instead of recomputing the forward pass during backpropagation, together with reusing per-thread buffers across episodes, gives a $1.48\times$ single-thread speedup. Removing the lock around the shared-network update, in the style of asynchronous SGD, gives $9.5\times$ on ten threads.

Robustness. The input guard $N \leq d(d-1) + 1$ was originally evaluated in 32-bit arithmetic and overflowed for large d ; evaluating it in 64-bit and bounding N fixes this. The command-line interface was fuzzed with degenerate and overflow inputs, all of which are now rejected cleanly, and the binaries were rebuilt with standard-library bounds assertions enabled without observing any out-of-range access. Two transient defects in early development versions, a discarded logit gradient and an incorrect entropy-bonus

sign, were corrected before the measurements above, and the corrected solver was validated against the reference solutions.

XI. CONCLUSION

This paper studied two parallel computational approaches to constructing optimal cyclic quorum systems, equivalently to finding difference covers. The first is a systematic recursive search with coverage-based pruning, reflection symmetry breaking, and partition-level parallelism. It is exact: it is guaranteed to find a solution whenever one exists in the search space, and it yields optimal base quorums through $N = 150$. The second is a deep reinforcement-learning agent that views the construction as a sequential decision problem and learns a policy by policy-gradient updates. It is not guaranteed to find a solution within a bounded number of episodes, but it extends the reachable range beyond the exhaustive one.

Future work includes generating k -ary necklaces with fixed content (where the count of each character is fixed) for $k > 2$, counting fixed-density prenecklaces, and developing efficient generators for other restricted classes of bracelets. More aggressive parallel search, or learned heuristics for

Algorithm 2 Reinforcement-learning episode and policy-gradient update

```

1: function EPISODE( $N, d, \pi_\theta$ )
2:    $s \leftarrow [\mathbf{0}_N, \mathbf{0}_N]$ ; store  $s_0$ 
3:   for  $k \leftarrow 1$  to  $d - 1$  do
4:      $z \leftarrow \pi_\theta(s)$ ;  $p \leftarrow \text{softmax}(z)$  with chosen numbers masked
5:     sample  $a_k \sim p$ 
6:      $r_k \leftarrow$  number of new differences covered by  $a_k$ 
7:     update  $s$  from  $a_k$ ; store  $s_k, a_k, r_k$ 
8:   end for
9:   return the trajectory  $(\dots, s_k, a_k, r_k, \dots)$ 
10: end function
11: function UPDATE( $\{r_k\}$ )
12:    $G_k \leftarrow \sum_{i>k} \gamma^{i-k} r_i$ ; normalise
13:    $\theta \leftarrow \theta + \eta \sum_k G_k \nabla_\theta \log \pi_\theta(a_k | s_k)$ 
14: end function

```

TABLE II

NEARLY OPTIMAL BASE QUORUMS FOUND BY THE REINFORCEMENT-LEARNING AGENT FOR $N = 151$ TO 171 .

N	Base quorum
151	0 3 7 34 48 59 60 80 82 103 121 122 126 132 134 141 150
152	0 4 7 10 61 62 69 80 81 99 102 116 118 125 128 130 141
153	0 1 5 6 12 40 60 62 80 88 95 98 109 112 128 131 140
154	0 16 21 27 31 40 48 62 73 77 98 116 125 127 128 141 147
155	0 6 24 40 41 53 64 80 83 101 115 120 126 131 146 151 153
156	0 10 22 39 72 81 88 90 102 105 109 116 141 143 146 147 148
157	0 2 14 15 19 30 52 67 102 103 109 110 112 132 133 153 155
158	0 12 16 23 29 33 34 37 60 69 79 84 99 110 112 117 138
159	0 25 53 58 65 77 80 84 97 98 100 106 108 118 147 148 154
160	0 2 9 10 22 41 64 65 75 79 90 93 106 109 114 115 123
161	0 21 37 38 39 45 56 58 60 67 71 81 131 143 146 151 158
162	0 13 14 15 16 40 50 53 55 59 63 67 78 139 144 146 155
163	0 20 25 36 71 81 85 93 101 102 104 107 119 136 143 156 160
164	0 9 29 56 67 68 82 86 88 89 114 119 129 130 137 154 160
165	0 9 19 31 58 63 80 82 122 124 128 130 133 137 151 158 164
166	0 3 11 19 31 59 66 73 76 83 116 133 139 143 144 146 148
167	0 17 18 20 41 49 54 63 79 86 94 96 98 100 146 156 158
168	0 2 15 39 42 60 70 76 77 81 86 106 117 127 129 135 149
169	0 10 31 44 56 74 76 87 94 96 99 100 115 136 144 152 166
170	0 22 36 43 45 46 51 70 91 103 107 111 120 131 138 141 164
171	0 4 17 25 27 36 56 60 72 100 106 118 121 122 145 159 164

pruning, may further extend the range of N for which optimal base quorums can be certified.

APPENDIX A

CONSTANT-AMORTIZED-TIME GENERATION OF
FIXED-DENSITY BRACELETS

In this appendix, k denotes the alphabet size, n the string length, and w the density (the number of non-zero symbols); these symbols are local to the appendix and are unrelated to the quorum parameters N and d of the main text.

The symmetry-breaking step of the combinatorial search enumerates **fixed-density bracelets**: the equivalence classes of length- n strings over a k -ary alphabet under rotation and reflection, restricted to strings with exactly w non-zero symbols. A k -ary **necklace** is the lexicographically least string in its rotation class and a **bracelet** the least string in its rotation-and-reflection class; a **prenecklace** is a prefix of some necklace. We write $N_k(n, w)$, $P_k(n, w)$, and $B_k(n, w)$ for the sets of fixed-density necklaces, prenecklaces, and bracelets. Efficiency is measured in the **constant**

amortized time (CAT) sense: the total work must be proportional to the number of objects produced, so that each successive object costs constant time on average.

Ruskey and Sawada gave the first CAT algorithm for generating fixed-density necklaces (Ruskey and Sawada 1999), and Sawada gave a CAT algorithm for bracelets (Sawada 2001); both build on the recursive necklace framework of Cattell, Ruskey, Sawada, Serra, and Miers (Cattell et al. 2000). The fixed-density necklace algorithm generates prenecklaces whose last character is non-zero, increasing the density by one at each recursive step rather than appending one character at a time, and it restricts the positions of the first and of subsequent non-zero characters. Its CAT property follows from bounding the size of its computation tree by a constant times $|N_k(n, w)|$.

Generating fixed-density bracelets is harder. The naive adaptation lists all fixed-density prenecklaces and tests each against its reversal, but a full reversal test costs $O(n)$ time and destroys the CAT bound. Two observations make

TABLE III

QUORUM SIZE d FOUND BY THE SEARCH COMPARED WITH THE LOWER BOUND $\lceil \sqrt{N} \rceil$ FOR SELECTED N . ROWS WITH $N \leq 150$ ARE OPTIMAL (RECURSIVE SEARCH); THE REST ARE NEARLY OPTIMAL (REINFORCEMENT LEARNING).

N	d	$\lceil \sqrt{N} \rceil$	$d/\lceil \sqrt{N} \rceil$
112	12	11	1.09
120	13	11	1.18
130	13	12	1.08
140	14	12	1.17
150	14	13	1.08
160	17	13	1.31
171	17	14	1.21

TABLE IV

COMPARISON OF THE TWO APPROACHES.

Feature	Recursive search	Reinforcement learning
Approach	Systematic, algorithmic search	Trial and error, from experience
Guarantee	Guaranteed, if the search is exhaustive	May not find a solution within the episode limit
Discovery	Explicit pruning rules	Strategies learned from rewards
Core logic	Recursive calls	A neural network

TABLE V

SUMMARY OF THE IMPLEMENTATION CHANGES AND THEIR MEASURED EFFECTS.

Change	Effect
Bracelet (reflection) symmetry	1.07–1.17 $\times$, identical output
Activation caching and buffer reuse	1.48 $\times$ (single thread)
Lock-free shared-network update	9.5 $\times$ (10 threads)

the test efficient. First, if a necklace begins with i equal characters followed by a different one, only reverse rotations beginning with the same prefix need be examined. Second, the decisive comparison between the suffix after the longest equal-to-reversal prefix and its reversal can be decided incrementally, by a reverse-status flag updated as the string is extended.

The fixed-density bracelet algorithm **BraceFD** merges these optimizations (Karim et al. 2014). Its correctness is stated by an initialization theorem: the enumeration lists every element of $B_k(n, w)$ exactly once in lexicographic order. For its CAT analysis, the computation tree has size proportional to $|B_k(n, w)|$, because $|N_k(n, w)|$ and $|B_k(n, w)|$ differ by at most a factor of two and the fixed-density necklace algorithm is already CAT. The only potentially expensive per-call operation is the reversal test. Each unequal comparison it performs is charged to the prenecklace being tested. For the equal comparisons one exhibits an injective mapping from a comparison to a distinct generated prenecklace—an explicit bijection for binary alphabets ($k = 2$) and a more involved mapping g for general k —so that the total number of equal comparisons is proportional to the number of prenecklaces and hence to $|B_k(n, w)|$. It follows that the total comparison cost is $O(|B_k(n, w)|)$, all other work per call is constant, and **BraceFD** is CAT, using linear space.

BIBLIOGRAPHY

- Bian, Kaigui et al. 2012. “Maximizing Rendezvous Diversity in Rendezvous Protocols for Decentralized Cognitive Radio Networks.” *IEEE Transactions on Mobile Computing* 12 (7): 1294–307.
- Bian, Kaigui, and Jung-Min Park. 2011. “Asynchronous Channel Hopping for Establishing Rendezvous in Cognitive Radio Networks.” *2011 Proceedings IEEE INFOCOM*, 236–40.
- Bian, Kaigui, Jung-Min Park, and Ruiliang Chen. 2009. “A Quorum-Based Framework for Establishing Control Channels in Dynamic Spectrum Access Networks.” *Proceedings of the 15th Annual International Conference on Mobile Computing and Networking*, 25–36.
- Bian, Yiming, and Arun K Somani. 2025. “CQS-Attention: Scaling up the Standard Attention Computation for Infinitely Long Sequences.” *IEEE Access*.
- Bouabdallah, Fatma, Raouf Boutaba, Ahmed Mehaoua, et al. 2018. “Collision Avoidance Energy Efficient Multi-Channel MAC Protocol for Underwater Acoustic Sensor Networks.” *IEEE Transactions on Mobile Computing* 18 (10): 2298–314.
- Cattell, Kevin, Frank Ruskey, Joe Sawada, Micaela Serra, and C. Robert Miers. 2000. “Fast Algorithms to Generate Necklaces, Unlabeled Necklaces, and Irreducible Polynomials over $\text{GF}(2)$.” *Journal of Algorithms* 37 (2): 267–82.
- Hall, Marshall. 1986. *Combinatorial Theory*. John Wiley & Sons.
- Huang, Tingpei, Bairen Zhang, Tiantian Zhang, Jianhang Liu, and Shibao Li. 2025. “QDND: Quorum-Based Energy Efficiency Aware Directional Neighbor Discovery in Ad Hoc Millimeter Wave Wireless Networks.” *International Journal of Communication Systems* 38 (6): e70005. <https://doi.org/https://doi.org/10.1002/dac.70005>.
- Jiang, Jehn-Ruey, Yu-Chee Tseng, Chih-Shun Hsu, and Ten-Hwang Lai. 2005. “Quorum-Based Asynchronous Power-Saving Protocols for IEEE 802.11 Ad Hoc Networks.” *Mobile Networks and Applications* 10: 169–81.
- Karim, S., Z. Alamgir, and S. M. Husnine. 2014. “Generating Fixed Density Bracelets of Arbitrary Base.” *International Journal of Computer Mathematics* 91 (3): 434–46.
- Kärkkäinen, Juha, Peter Sanders, and Stefan Burkhardt. 2006. “Linear Work Suffix Array Construction.” *Journal of the ACM (JACM)* 53 (6): 918–36.
- Kleinheksel, Cory J., and Arun K. Somani. 2016. “Scaling Distributed All-Pairs Algorithms.” *Proceedings of the International Conference on Information Science and Applications*, 247–57.
- Kleinheksel, Cory J., and Arun K. Somani. 2018. “Efficient Distributed All-Pairs Algorithms: Management Using Optimal Cyclic Quorums.” *IEEE Transactions on Parallel and Distributed Systems* 29 (2): 391–404.
- Lai, Shouwen, Binoy Ravindran, and Hyeonjoong Cho. 2010. “Heterogenous Quorum-Based Wake-up Scheduling in Wireless Sensor Networks.” *IEEE Transactions on Computers* 59 (11): 1562–75.
- Luk, Wai-Shing, and Tien-Tsin Wong. 1997. “Two New Quorum Based Algorithms for Distributed Mutual Exclusion.” *Proceedings of the 17th International Conference on Distributed Computing Systems*, 100–106.
- Maekawa, Mamoru. 1985. “A $\sqrt{N}$ Algorithm for Mutual Exclusion in Decentralized Systems.” *ACM Transactions on Computer Systems* 3 (2): 145–59.
- Mehnert, Pascal. 2024. “Scalable Distributed String Sorting Algorithms.” PhD thesis, Karlsruher Institut für Technologie (KIT).
- Pozza, Riccardo, Michele Nati, Stylianos Georgoulas, Klaus Moessner, and Alexander Gluhak. 2015. “Neighbor Discovery for Opportunistic Networking in Internet of Things Scenarios: A Survey.” *IEEE Access* 3: 1101–31.
- Ruskey, F., and J. Sawada. 1999. “An Efficient Algorithm for Generating Necklaces of Fixed Density.” *SIAM Journal on Computing* 29 (6): 1969–85.
- Sawada, J. 2001. “Generating Bracelets in Constant Amortized Time.” *SIAM Journal on Computing* 31 (1): 259–68.
- Saxena, Prem Chandra, and Jagmohan Rai. 2003. “A Survey of Permission-Based Distributed Mutual Exclusion Algorithms.” *Computer Standards & Interfaces* 25 (2): 159–81.